

Proyecto Árboles AVL

Introducción.

La idea principal de este proyecto es: armar un sistema de inventario de una tienda de libros ficticia, implementando árboles de búsqueda binaria y árboles AVL (para evitar tener árboles desbalanceados). La razón por la cual un árbol para este sistema es más eficiente que algún tipo de lista o diccionario de Python, es que para la tarea de filtrar **por límites inferiores o superiores (o ambos)**, podemos desarrollar esta tarea en un número de comparaciones muy inferior a realizar la tarea con otra estructura de datos.

Implementación.

Dedicamos un script entero a la implementación de cuatro clases, que van a ser troncales en el resto del proyecto:

1. **Clase Nodo:** Guarda pares (*nombre*, *valor*), los cuales por el atributo **valor** va a depender la organización de los nodos y **nombre** es meramente una referencia a ese nodo.
2. **Clase BST:** Implementa la estructura de dato de Árbol de Búsqueda Binaria, tiene su método de inserción de nuevos nodos (aplicado con **recursividad**), un método de eliminación de nodos (que actúa de manera distinta, según si el nodo a eliminar tiene hijos o no, etc.).
3. **Clase Nodo AVL:** Clase que hereda otros atributos de la clase Nodo anterior. En esta clase es donde implementamos el **atributo altura**, que nos va a ayudar para hacer el proceso de **rebalanceo del árbol**.
4. **Clase AVL:** Hereda de la clase **BST**, y sobrescribe los métodos de inserción de nodos e implementa métodos para rotar subárboles, para poder hacer funcional la **acción de balancear el árbol**.

Extracción de Datos.

Los datos que utilizo para este proyecto, los obtuve de la página web de *HuggingFace*, encontré un set de datos con casi 7 millones de datos, con observaciones de libros registrados en la base de datos de *Goodreads*, que es una página/app, que la gente puede calificar los libros que leyó y les hace reseñas.

De aquella base de datos me extraigo los datos de:

- **Título del Libro**
- **Nombre del Autor**
- **Género Principal**
- **Estrellas en la plataforma (1-5 Estrellas)**
- **Cantidad de Reseñas** → (*Este dato me lo traje para medir la "popularidad" de un libro*)

Indexación por Claves Compuestas

Para generar los valores correspondientes a cada libro, debía optar por dos opciones:

- Darle varios valores a cada nodo, uno por cada columna
- Construir una clave compuesta que encapsule todos los atributos en un solo número.

Me decidí por usar una clave compuesta, asignando el siguiente orden de valores:

$$Clave = id_{Estrellas} \times 10^{23} + id_{Titulo} \times 10^{16} + id_{Género} \times 10^{12} + id_{Autor} \times 10^5 + id_{N-Reseñas} \times 10^0$$

Aplicación del Proyecto:

Es importante, antes de poder probar el producto final, debemos tener unas tablas, por los cuales indexo y determino el "id" de cada atributo para cada nodo. Esto es un procedimiento que tarda aproximadamente 10 minutos, esto se debe que estoy usando el set de datos en remoto, desde el repositorio que lo encontré, porque, de otra manera, pesaba aproximadamente 8 gigabytes de memoria. Debemos correr el archivo *data_handling.py*.

- **Uso del archivo "create_tree.py".**

Tenemos varias funcionalidades en el script, donde podemos jugar con las funciones y ajustar los parámetros. La función *create_avl_tree*, creamos el árbol AVL y podemos determinar cuantas observaciones no queremos traer desde el set de datos remoto. Luego, podemos implementamos un método que busca entre rangos (cotas mínimas y máximas) para el parámetro prioritario, (en esta configuración del proyecto, podemos filtrar por cotas mínimas y máximas las estrellas del libro). Para ello tuve que **definir nuevos métodos en mis clases de las estructuras de datos**, para hacer un filtrado entre dos límites.

Futuras Mejoras.

Para mejorar el producto, se podrían crear métodos que filtren por los otros atributos no-prioritarios de cada nodo, pero eso perdería la capacidad de búsqueda eficiente que tienen los árboles, porque no podíamos descartar ramas enteras, tendríamos que hacer una búsqueda $O(n)$ a todo el árbol. Como también, se podrían implementar funciones de poder seleccionar libros en una interfaz gráfica para poder eliminar del árbol determinados libros, simulando el proceso de "quedarnos sin stock".